Rupeek AI Testing — Fully Agentic Automation Plan

Sep 22, 2026 · @Piyush Kumar

Executive Summary

Where We Are Today

Your AI testing system is already ahead of traditional automation. Claude dynamically identifies selectors, evaluates assertions, and self-heals failures — no brittle XPaths. But **humans are still required at 4 points in every test cycle**:

Manual Step	Frequency	Who Does It
Write JSON flow configs (actions + assertions per step)	Every new feature or UI change	Engineer
Trigger test runs	After every deploy	QA / Engineer
Enter OTP during auth step	Every single run	QA / Engineer
Review heal logs and fix stuck flows	When Claude cannot self-heal	QA Lead

What "Fully Agentic" Means

Fully agentic = **zero humans in the loop for routine testing cycles**. Humans get involved only when there is a genuine product bug to fix. Everything else — config writing, run triggering, OTP fetching, failure healing, visual checks, and reporting — happens automatically.

Goal State After This Plan

```
Code pushed to GitHub
        ↓
Explorer agent visits app → writes / updates test configs automatically
        ↓
Tests auto-trigger on schedule + on every deploy
        ↓
OTP fetched from backend API automatically
        ↓
Failures self-healed by Claude (already works today)
        ↓
Slack alert ONLY when there is a real product bug
        ↓
Human looks at only the bug — not the test infrastructure
```

Estimated manual effort reduction: ~85% per sprint.

System Architecture — After Full Agentic Upgrade

```
flowchart TD
    A[GitHub Push / PR] --> B[GitHub Actions Workflow]
    B --> C[Explorer Agent - Stagehand]
    C --> D{Config drift?}
    D -- Yes --> E[Auto-update JSON flow configs]
    D -- No --> F[Skip config update]
    E --> G[Auto-start Test Run via REST API]
    F --> G
    G --> H[node-cron Scheduled Run]
    H --> G
    G --> I[Test Engine - NestJS]
    I --> J[Browser - Playwright]
    J --> K[Claude - Selector Planner]
    K --> J
    J --> L{Step passes?}
    L -- Yes --> M[Next Step]
    L -- No --> N[Claude - Diagnose Failure]
    N --> O{Recovery type}
    O -- retry --> J
    O -- patch_config --> E
    O -- skip_action --> M
    O -- abort --> P[Mark step FAIL]
    M --> Q{All steps done?}
    Q -- No --> J
    Q -- Yes --> R[Visual Screenshot Comparison]
```

```
    R --> S[Claude - Generate Report Summary]
    S --> T{Any failures?}
    T -- No --> U[Slack: All Clear]
    T -- Yes --> V[Slack: Failure Alert + Screenshots]
    V --> W[GitHub Issue Auto-created]
```

Key Components After Upgrade

Component	Technology	Status
Browser automation	Playwright	Already built
AI selector planning	Claude via Anthropic SDK	Already built
Self-healing engine	Claude diagnoseFailure	Already built
Config patching	HealerService patch_config	Already built
REST API for runs	NestJS /runs controller	Already built
Slack notifications	fetch to webhook	Already built
Explorer agent	**Stagehand + Claude**	**To build**
Scheduled runs	**node-cron**	**To build (trivial)**
CI/CD triggers	**GitHub Actions**	**To build**
Auto OTP	**API OTP as default**	**To build (config change)**
Visual regression	**Playwright toHaveScreenshot**	**To build**
Auto GitHub issues	**GitHub REST API**	**To build**

Phase 1 — Zero Manual Triggers

Goal: No human should ever have to type a command to start a test run or enter an OTP.

Effort: ~2–3 days. Mostly config wiring, no new architecture.

1A. Wire the Schedule Field to node-cron

Your `TestRules.schedule` field already holds a cron string (e.g. `"0 */6 * * *"`) in every config file. It is never consumed today. At app startup, the module reads each variant's schedule string and registers a cron job internally — no external cron server or deployment change needed.

Install: `node-cron` + `@types/node-cron` (two npm packages, ~10KB, no new services).

Result: Tests run on schedule automatically. No Slack message, no cron job to set up on a server. The app manages its own schedule.

1B. Make API OTP the Default

Today `TerminalOtpProvider` is the default — it waits for a human to type. `ApiOtpProvider` already exists but is only used when `RUN_CONTEXT=api`. Flip the default:

File to change: `src/modules/otp/otp.module.ts` — the provider factory that decides which OTP strategy to use at runtime.

Change the provider registration so `ApiOtpProvider` is default when `INTERACTIVE_OTP=false`. Required env vars to set: `TEST_OTP`, `INTERACTIVE_OTP=false`, `RUN_CONTEXT=api`. Terminal prompt stays active only in local dev.

Result: OTP is fetched from your backend API automatically. No human sits waiting.

1C. GitHub Actions CI Workflow

Add `.github/workflows/test.yml` with three triggers: push to main/staging, pull_request to main, and a daily 2am schedule. The workflow starts the NestJS server, calls the `/runs` REST API to kick off a test run, polls until completion, and uploads the HTML report as an artifact. Exit code mirrors the number of failed steps.

GitHub secrets to add: `ANTHROPIC_API_KEY` and `TEST_OTP` — both already exist in your local `.env`; just register them in the repo's Settings → Secrets.

Result: Every push to main/staging auto-runs tests. PR checks show pass/fail. No human trigger needed.

Phase 2 — Flow Auto-Generation with Stagehand

Goal: No engineer should ever hand-write JSON flow configs. An AI explorer visits the live app and generates them automatically.

Effort: ~1–2 weeks. This is the highest-impact change in the whole plan.

What Is Stagehand?

Stagehand is an open-source TypeScript library built on top of Playwright (your exact stack) that adds three AI-powered primitives:

Method	What it does
`page.act("click the gold loan card")`	Performs a natural language action — Claude figures out the selector
`page.observe()`	Returns all interactable elements on the page with plain-English descriptions
`page.extract({ schema })`	Extracts structured data from the page into a typed object

This is essentially a smarter version of your existing `agent.planStep()` — but it maintains page context across steps and handles edge cases (shadow DOM, dynamic content, iframes) better.

The Explorer Agent — How It Works

Create a new CLI command: `npm run explore -- --variant fresh`

This explorer agent will:

1. Launch the app at `site.baseUrl`
2. Use `page.observe()` at each screen to see all interactable elements
3. Perform the action using `page.act()` with natural language
4. Record every action + assertion as a `FlowStep` JSON object
5. Write the output to `config/test-rules-fresh.config.json` automatically

New module to build: `src/modules/explorer/` — contains the ExplorerService, its NestJS module, and a CLI entry point (`npm run explore`). The service takes a variant name, visits the live app using Stagehand, and writes the generated flow steps back to the corresponding config JSON.

Integration Points

GitHub Actions integration: Add an explorer step before the test run step in the CI workflow. If the explorer detects any config drift, it commits the updated JSON to the branch automatically. This means the test run that follows always uses the most current version of the app flow.

Result: When the Rupeek app UI changes, the explorer detects the drift, updates the config, and commits the fix — all without a human writing a single line of JSON.

Phase 3 — Visual Regression Testing

Goal: Automatically detect if the UI looks different from the last known-good run — without anyone manually comparing screens.

Effort: ~2–3 days. Built entirely on Playwright's built-in screenshot diffing — zero new dependencies.

How It Works

Playwright's `toHaveScreenshot()` compares a live screenshot against a stored baseline PNG. If pixels differ beyond a threshold, the step fails automatically. Baselines are committed to the repo and updated only when a human approves the visual change.

Where to Add It

Your `TestAssertion` type today supports `url_contains`, `element_visible`, `text_matches`, and `no_error_toast`. Add a fifth type — **`visual_match`** — that plugs into the existing assertion evaluation loop without touching the rest of the flow.

Two files need changes: the type definition and the assertion handler in FlowStepService. Both are small, targeted additions — no refactoring of existing logic.

Change needed in `src/common/types/test.types.ts` — add `visual_match` to the `AssertionType` union and add an optional `maskSelectors` field to `TestAssertion` (used to mask dynamic price areas during pixel diffing).

Change needed in `src/modules/flow-step/flow-step.service.ts` — add a `visual_match` handler in the assertion block. It calls Playwright's built-in `toHaveScreenshot()` with the assertion label as the filename, a pixel tolerance of ~100px, and optional mask selectors to exclude dynamic content like loan amounts.

Adding Visual Assertions to Your Config

In any `test-rules-*.config.json`, add `visual_match` assertions to steps where UI must not regress — typically the offer page, booking confirmation, and auth modal. The assertion label becomes the baseline filename stored in `tests/snapshots/`.

Baseline Workflow

Baselines are PNG screenshots stored in `tests/snapshots/` and committed to git like any test fixture. On the first run, generate them with `--update-snapshots`. After a deliberate UI redesign ships, update baselines by running with `--update-snapshots` and

committing the new PNGs. Every subsequent CI run compares against the committed baseline automatically — no command needed.

What Gets Caught Automatically

- Font or colour changes in the offer page
- Layout shifts after a CSS change
- A component moving 10px due to a refactor
- A missing section that HTML still renders (but visually blank)

None of these would be caught by your current `element_visible` or `url_contains` assertions alone.

Phase 4 — Autonomous Maintenance

Goal: When something breaks that Claude cannot self-heal, the system creates a GitHub issue automatically — no human has to check Slack, find the logs, and file a ticket.

Effort: ~3–4 days.

4A. Auto-Create GitHub Issues on Unrecoverable Failures

When a run ends with `status: fail` and the heal log shows `outcome: failed` (Claude tried but could not recover), automatically open a GitHub issue with the full context.

New service to build: `src/modules/notifications/github-issue.service.ts` — called at the end of a run when failures have `outcome: failed` in the heal log. It posts a structured issue to GitHub with the run ID, site name, Claude's summary, failed step names, and errors. Labels the issue `automated-test-failure` and `needs-triage` automatically.

Wire into `TestEngineService.runTestSuite()` after the existing Slack notification block. Call `createFailureIssue()` only when the heal log contains attempts with `outcome: failed` — meaning Claude tried and could not recover.

4B. Richer Slack Alerts

Upgrade `sendSlackNotification()` in `TestEngineService` to include: number of self-healed steps, number of hard failures, Claude's plain-English summary of root causes, and a direct link to the HTML report. This gives the on-call person full context without opening any logs.

- Screenshot of the failed step (attach as file via Slack Files API)

- Direct link to the auto-generated HTML report
- Which steps self-healed vs which hard-failed
- Claude's summary of the root cause

4C. Config Drift Detection on Schedule

Add a weekly Monday morning cron job (separate from the test-run schedule) that runs only the explorer agent — not the full test suite. If drift is found, configs are auto-updated and a Slack message goes out asking the team to review the auto-commit. This is lighter than a full run and catches UI changes before they cause test failures mid-week.

Result: Every Monday morning, the system checks if the Rupeek app UI has changed and auto-updates configs. The team gets a Slack heads-up to review the auto-commit, not to do the work themselves.

File-by-File Implementation Guide

Every file you need to create or modify, in execution order.

Files to CREATE (new)

File	Phase	What it does
`.github/workflows/test.yml`	1	CI/CD — triggers runs on push, PR, and schedule
`src/modules/explorer/explorer.service.ts`	2	Stagehand-powered flow explorer agent
`src/modules/explorer/explorer.module.ts`	2	NestJS module wiring for explorer
`src/modules/explorer/explorer.cli.ts`	2	CLI entry point: `npm run explore`
`src/modules/notifications/github-issue.service.ts`	4	Auto-creates GitHub issues on unrecoverable failures
`tests/snapshots/`	3	Directory for Playwright visual baseline PNGs (git-committed)

Files to MODIFY (existing)

`src/main.ts` — Phase 1A

- Add `node-cron` schedule loop after app bootstrap
- Read `rules.schedule` from each variant config and register cron jobs

`src/modules/otp/otp.module.ts` — Phase 1B

- Change default OTP provider from `TerminalOtpProvider` to `ApiOtpProvider` when `INTERACTIVE_OTP=false`
- Terminal provider only active in local dev

`src/common/types/test.types.ts` — Phase 3

- Add `'visual_match'` to `AssertionType` union
- Add optional `maskSelectors?: string[]` to `TestAssertion` interface (to mask dynamic price areas during visual diff)

`src/modules/flow-step/flow-step.service.ts` — Phase 3

- Add `visual_match` handler in the assertion evaluation block
- Call `expect(page).toHaveScreenshot(name, options)` for this assertion type

`src/modules/test-engine/test-engine.service.ts` — Phase 4

- Inject `GithubIssueService`
- After Slack notification: call `createFailureIssue()` if unrecoverable failures exist
- Upgrade `sendSlackNotification()` to include healed count, hard-fail count, report link, Claude summary

`src/app.module.ts` — Phase 2 + 4

- Register `ExplorerModule` and `GithubIssueService`

`.env` / **`.env.example`** — Phase 1 + 4

- Add: `INTERACTIVE_OTP=false`, `RUN_CONTEXT=api`, `GITHUB_TOKEN`, `GITHUB_REPO`, `REPORT_BASE_URL`

`config/test-rules-*.config.json` — Phase 3

- Add `visual_match` assertions to key steps (offer page, booking confirmation, auth modal)

`package.json` — Phase 1 + 2

- Add script: `"explore": "ts-node src/modules/explorer/explorer.cli.ts"`
- Add script: `"start:prod": "node dist/main"`

Implementation Order (do this sequence)

1. `.env` changes + OTP module default flip — Phase 1B (30 min)
2. Schedule wiring in app startup — Phase 1A (1 hour)
3. GitHub Actions workflow file — Phase 1C (2 hours)
4. Add `visual_match` assertion type + handler — Phase 3 (half day)
5. Explorer service + CLI command — Phase 2 (3–4 days)
6. GitHub issue auto-creation service — Phase 4A (half day)
7. Slack notification upgrade — Phase 4B (half day)
8. Config drift weekly cron — Phase 4C (2 hours)

Phases 1 and 3 can be done in parallel. Phase 2 is independent and the longest item.

Dependencies to Install

All open source. No paid services. Uses your existing `ANTHROPIC_API_KEY`.

Package	Version	Phase	Purpose
`node-cron`	`^3.0.0`	1	Schedule test runs from inside the NestJS app
`@types/node-cron`	`^3.0.0`	1	TypeScript types for node-cron
`wait-on`	`^7.0.0`	1	GitHub Actions: wait for NestJS to be ready before running tests
`@browserbasehq/stagehand`	`^1.0.0`	2	Explorer agent — AI-powered Playwright layer
`zod`	`^3.0.0`	2	Schema validation for Stagehand `extract()` calls

What You Already Have (no reinstall needed)

Package	Already does
`@anthropic-ai/sdk`	Claude API calls — Stagehand reuses this
`playwright`	Browser automation — Stagehand is built on top of it
`@nestjs/*`	Framework — all new modules follow existing patterns

Package	Already does
`typescript`	All new files are plain TypeScript

No New Paid Services

- Stagehand runs locally with `env: 'LOCAL'` — no Browserbase cloud account needed
- GitHub Actions is free for public repos; 2,000 min/month free for private repos
- Visual regression uses Playwright's built-in diffing — no Applitools or Percy
- GitHub Issues API is free with a personal access token
- node-cron runs inside your existing NestJS process — no separate cron server

What Stays Manual — The Honest 10%

Full automation is real but not absolute. Here is what will always need a human, and why that is fine.

Situation	Why It Cannot Be Automated	Human Action Needed
Genuine product bug	The app is broken — no automation can fix the product	Engineer fixes the bug
First-ever new flow (e.g. a completely new product page)	Explorer needs the page to exist first; cannot test what is not live	Run `npm run explore` once after the page goes live
OTP provider is down	If your backend OTP API is unavailable, no fallback exists	Temporarily set `INTERACTIVE_OTP=true` and run locally
Visual baseline approval	When a deliberate UI redesign ships, baselines must be updated by a human who confirms the new design is correct	Run `playwright test --update-snapshots`, review, commit
Claude heal log review	When Claude patches a config with `confidence: medium`, a human should spot-check the patch is correct	Review `heal-log.json` in the run report weekly

Situation	Why It Cannot Be Automated	Human Action Needed
GitHub Actions secret rotation	`ANTHROPIC_API_KEY` and `TEST_OTP` must be rotated periodically	Update GitHub repo secrets

---

Why This 10% Is Acceptable

Every item in the table above is either:

1. **A real product problem** — something is genuinely broken and a human must fix it. No amount of test automation changes this.
2. **A one-time setup step** — not a recurring burden. Once done, it does not come back.
3. **A deliberate human decision** — approving a visual change or a config patch requires human judgment by design. Automating this would make the system override its own correctness guarantees.

The goal is not zero human involvement. The goal is that humans spend time on decisions, not on operations. After this plan, every human touchpoint is a decision — not a chore.

Summary: Before vs After

Activity	Before	After
Starting a test run	Manual command every time	Automatic on push + on schedule
Writing test configs	Hand-coded JSON	Auto-generated by explorer agent
Entering OTP	Human waits and types	API fetches it automatically
Reacting to a failure	Check Slack, read logs, file ticket	GitHub issue auto-created with full context
Detecting UI regressions	Manual visual review	Playwright pixel-diff catches it automatically
Config drift when app changes	Nobody notices until tests break	Weekly explorer diff + auto-commit + Slack alert
Human time per week on test ops	~4–6 hours	• ~30 minutes (reviewing auto-decisions only)